

MICROSAR EthTSyn

Technical Reference

Global Time Synchronization over Ethernet

Version 13.1.0

Authors	Jeroen Laverman, Michael Seidenspinner, Fabian Ziegler
Status	Released

Document Information

History

Author	Date	Version	Remarks
Jeroen Laverman	2014-09-23	1.0.0	Creation of document
Jeroen Laverman	2014-11-11	1.0.1	Add new API
Michael Seidenspinner	2015-01-12	1.0.2	ESCAN00080452 Modification of Chapter "Configuration"
Michael Seidenspinner	2015-02-26	1.1.0	Added Configuration for SW-Timestamping
Michael Seidenspinner	2015-07-10	1.2.0	ESCAN00083936
Michael Seidenspinner	2015-10-23	2.0.0	ESCAN00085377: FEAT-1529: Support Ethernet Switches for Ethernet Time Sync
Michael Seidenspinner	2016-03-15	2.1.0	ESCAN00085303 Added Chapters: > Boundary Clock > AlwaysAsCapable > Announce > Source Port Identity Check > Correction Action
Michael Seidenspinner	2016-05-13	2.2.0	ESCAN00089686 Added Chapter: > Flexible Pdelay configuration
Michael Seidenspinner	2016-11-25	3.0.0	FEATC-248: FEAT-1998: Support of HW Time Stamping for Switch for EthTSyn (SysService_AsrTSynEth)
Michael Seidenspinner	2017-03-07	4.0.0	FEATC-383: FEAT-2279: Time Synchronization acc. AR 4.3 for EthTSyn
Michael Seidenspinner	2017-04-12	4.0.1	Fixed review findings
Michael Seidenspinner	2017-07-05	5.0.0	STORYC-1213, STORY-128
Michael Seidenspinner	2018-02-22	6.0.0	Formal rework
Michael Seidenspinner	2018-08-14	6.0.1	Fixed review findings
Michael Seidenspinner	2018-11-16	7.0.0	STORYC-6413: [AVB] Implementation of 802.1Qbv extensions for EthTSyn
Michael Seidenspinner	2019-01-31	7.0.1	Added EthIf restriction for STORYC-6413

Michael Seidenspinner	2019-01-31	8.0.0	Added Switch management limitations
Michael Seidenspinner	2019-03-22	9.0.0	Adapted SyncSentCbk and FollowUpSentCbk
Michael Seidenspinner	2019-07-25	10.0.0	Switch management API acc. to AUTOSAR 4.4
Michael Seidenspinner	2019-11-22	11.0.0	ESCAN00104993
Michael Seidenspinner	2020-04-03	12.0.0	StbM interaction acc. to AUTOSAR 4.4
Michael Seidenspinner, Fabian Ziegler	2020-08-04	12.1.0	Time validation, Optimization of switch time synchronization algorithm, Bridge modes Transparent/Boundary clock, Switch and Host as one time-aware system
Michael Seidenspinner	2020-10-02	13.0.0	Updated documentation after code refactoring
Fabian Ziegler	2020-12-14	13.1.0	Support of debounce time

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_TimeSyncOverEthernet.pdf	V4.4.0
[2]	AUTOSAR	AUTOSAR_SWS_TimeSyncOverEthernet.pdf	V19-11
[3]	AUTOSAR	AUTOSAR_SWS_DefaultErrorTracer.pdf	V4.3.0
[4]	AUTOSAR	AUTOSAR_SWS_DiagnosticEventManager.pdf	V4.3.0
[5]	AUTOSAR	AUTOSAR_SWS_SynchronizedTimeBaseManager.pdf	V4.4.0
[6]	AUTOSAR	AUTOSAR_SWS_SynchronizedTimeBaseManager.pdf	V19-11
[7]	IEEE	IEEE 802.1AS-2011: Timing and Synchronization for Time-Sensitive Applications in Bridged Local Networks	2011
[8]	IEEE	IEEE 1588-2008: IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems	2008
[9]	Vector	TechnicalReference_IpBase.pdf	see delivery
[10]	Vector	TechnicalReference_EthIf.pdf	see delivery
[11]	Vector	TechnicalReference_StbM.pdf	see delivery

Scope of the Document

This technical reference describes the general use of the EthTSyn basis software. The EthTSyn can only be used in conjunction with the StbM (see [5]), the EthIf (see [10]) and the Eth basis software module which is also part of the delivery.

**Caution**

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1	Component History	10
2	Introduction.....	11
2.1	Architecture Overview	12
3	Functional Description	13
3.1	Features	13
3.1.1	IEEE 802.1AS-2011 Conformance	13
3.1.2	General Limitations	13
3.1.3	IEEE 802.1AS-2011 Deviations.....	13
3.1.4	AUTOSAR 4.4.0 Deviations	14
3.1.5	Switch management limitations	14
3.1.5.1	Message limitations	14
3.1.5.2	AUTOSAR message format limitations	14
3.1.5.3	Slave port limitations.....	14
3.1.5.4	Master port limitations.....	14
3.1.6	Time-Aware-Bridge (Switch Management)	15
3.1.6.1	Switch Timestamping.....	19
3.1.6.2	Switch and Host as one Time-Aware System.....	19
3.1.6.3	Time-Aware-Bridge behavior.....	19
3.1.6.4	Time-Aware-Bridge (Transparent Clock)	20
3.1.6.4.1	Sync and FollowUp message reception	20
3.1.6.4.2	Sync and FollowUp message transmission.....	20
3.1.6.4.3	Sync timeout handling.....	21
3.1.6.4.4	Source port identity modification	21
3.1.6.5	Time-Aware-Bridge (Boundary Clock).....	21
3.1.6.5.1	Sync and FollowUp message reception	21
3.1.6.5.2	Sync and FollowUp message transmission.....	22
3.1.6.5.3	Sync timeout handling.....	22
3.1.6.5.4	Source port identity modification	22
3.1.6.6	Bridge as Grand Master.....	22
3.1.6.7	Pdelay.....	22
3.1.7	Clock Master Role.....	22
3.1.7.1	Immediate time synchronization.....	23
3.1.8	Clock Slave Role.....	23
3.1.8.1	Announce	23
3.1.8.2	Source Port Identity Check	24
3.1.9	Path Delay (Pdelay) Measurement.....	24

3.1.9.1	Flexible Pdelay configuration	24
3.1.10	AlwaysAsCapable	26
3.1.11	Acting as Time-Master and Time-Slave in parallel.....	26
3.1.12	Debounce time	26
3.1.13	Time synchronization of ethernet switches	27
3.1.13.1	Cascaded switches	27
3.1.13.2	Notification about changes in the switch synchronization state.....	28
3.1.13.3	Rate regulator time synchronization mechanism.....	28
3.1.13.3.1	Initial offset correction	30
3.1.13.3.2	Offset jump correction threshold	30
3.1.14	Time validation	30
3.2	Interaction with EthIf and StbM BSW module.....	31
3.3	Initialization	31
3.4	States	32
3.5	Main Functions	32
3.6	Error Handling.....	33
3.6.1	Development Error Reporting.....	33
3.6.2	Runtime Error Reporting	34
3.6.3	Production Code Error Reporting	34
4	Integration.....	35
4.1	Scope of Delivery.....	35
4.1.1	Static Files	35
4.1.2	Dynamic Files	36
4.2	Critical Sections	36
4.2.1	Exclusive Area 0	36
4.2.2	Exclusive Area Get Ts	38
4.2.3	Exclusive Area Time Valid	39
4.3	Main Functions	40
5	API Description.....	42
5.1	Type Definitions	42
5.2	Services provided by EthTSyn	42
5.2.1	EthTSyn_GetVersionInfo.....	42
5.2.2	EthTSyn_Init	43
5.2.3	EthTSyn_InitMemory.....	43
5.2.4	EthTSyn_MainFunction.....	44
5.2.5	EthTSyn_SwtMgmt_MainFunction	44
5.2.6	EthTSyn_SetTransmissionMode	45
5.3	Services used by EthTSyn	45

5.4	Callback Functions.....	47
5.4.1	EthTSyn_RxIndication.....	47
5.4.2	EthTSyn_TxConfirmation	48
5.4.3	EthTSyn_TrcvLinkStateChg	49
5.5	Configurable Interfaces	49
5.5.1	Notifications	49
5.5.1.1	SyncSentCbk	49
5.5.1.2	FollowUpSentCbk	50
5.5.1.3	SwitchSyncStateChangeCbk	50
6	Configuration	52
6.1	Configuration Variants.....	52
6.2	Configuration of switch time synchronization.....	52
7	Glossary and Abbreviations	54
7.1	Glossary	54
7.2	Abbreviations	54
8	Contact.....	55

Illustrations

Figure 2-1	AUTOSAR 4.3 Architecture Overview	12
Figure 2-2	Interface to adjacent modules of EthTSyn	12
Figure 3-1	Bridge Overview	16
Figure 3-2	Sequence diagram of the switch management reception path	17
Figure 3-3	Sequence diagram of the switch management transmission path	18
Figure 3-4	Sequence diagram of path delay measurement	24
Figure 3-5	Schematic representation of the measurement sequence used by the rate regulator time synchronization mechanism	29
Figure 6-1	Configuration of switch time synchronization	53
Figure 6-2	Configuration of rate regulator	53

Tables

Table 1-1	Component history	10
Table 3-1	Feature conformance to IEEE 802.1AS-2011	13
Table 3-2	General Limitations	13
Table 3-3	Deviations to see IEEE 802.1AS-2011: Timing and Synchronization for Time-Sensitive Applications in Bridged Local Networks	14
Table 3-4	Comparison of Transparent Clock and Boundary Clock	20
Table 3-5	General Pdelay configuration options	25
Table 3-6	Pdelay initiator configuration options	25
Table 3-7	Reported switch sync states	28
Table 3-8	EthTSyn state machines	32
Table 3-9	EthTSyn_MainFunction() description	32
Table 3-10	EthTSyn_SwtMgmt_MainFunction() description	33
Table 3-11	Service IDs: Development error reporting	33
Table 3-12	Development errors reported to DET	34
Table 3-13	Service IDs: Runtime error reporting	34
Table 3-14	Runtime errors reported to DET	34
Table 3-15	Errors reported to DEM	34
Table 4-1	Static files	36
Table 4-2	Generated files	36
Table 4-3	Exclusive Area 0	38
Table 4-4	Exclusive Area Get Ts	39
Table 4-5	Exclusive Area Time Valid	40
Table 5-1	Type definitions	42
Table 5-2	EthTSyn_GetVersionInfo	42
Table 5-3	EthTSyn_Init	43
Table 5-4	EthTSyn_InitMemory	43
Table 5-5	EthTSyn_MainFunction	44
Table 5-6	EthTSyn_SwtMgmt_MainFunction	44
Table 5-7	EthTSyn_SetTransmissionMode	45
Table 5-8	Services used by the EthTSyn	46
Table 5-9	EthTSyn_RxIndication	47
Table 5-10	EthTSyn_TxConfirmation	48
Table 5-11	EthTSyn_TrcvLinkStateChg	49
Table 5-12	SyncSentCbk	50
Table 5-13	FollowUpSentCbk	50
Table 5-14	SwitchSyncStateChangeCbk	51
Table 7-1	Glossary	54
Table 7-2	Abbreviations	54

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.xx	Created Beta version
1.01.xx	SW-Timestamp support
2.00.xx	Time-Aware-Bridge support
2.01.xx	Boundary Clock support
2.02.xx	Flexible Pdelay configuration
3.00.xx	Switch Timestamp support
4.00.xx	AR 4.3 Support
5.00.xx	Immediate Time Sync support
5.01.xx	P3 release
6.00.xx	Master-Slave conflict detection
6.01.xx	Refactoring for SafeBSW
6.02.xx	Support time base specific time difference calculation acc. ASR4.3.1
7.00.xx	Switch time synchronization
8.00.xx	MISRA-C:2012
9.00.xx	ASIL_D, signature change of SyncSentCbk and FollowUpSentCbk
10.00.xx	Switch management API acc. to AUTOSAR 4.4
11.00.xx	PdelayLatencyThreshold acc. to AUTOSAR 4.4
12.00.xx	Time synchronization acc. to AUTOSAR 4.4
12.01.xx	Time validation for Bridge, Switch and host as one Time-Aware-System (BETA)
12.02.xx	Time validation for EndStation (BETA)
12.03.xx	Bridge operation modes: Transparent and Boundary Clock (BETA)
12.04.xx	Release of Features introduced with Versions 12.01.xx-12.03.xx, Optimization of switch time synchronization algorithm
13.00.xx	Support Switches in ASIL Ethernet
13.01.xx	Support of post build selectable variance of EthIf
13.02.xx	Support of debounce time feature

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module EthTSyn as specified in [1].

Supported AUTOSAR Release*:	4.4.0	
Supported Configuration Variants:	Pre-compile	
Vendor ID:	EthTSyn_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	EthTSyn_MODULE_ID	164 decimal (according to ref. [1])

* For the precise AUTOSAR Release 4.x please see the release specific documentation.

The EthTSyn module provides the functionality of time synchronization defined by the gPTP (generalized Precision Time Protocol) IEEE 802.1AS-2011 (see [7]). The gPTP uses Ethernet communication for transmission of time stamped Ethernet frames to achieve time synchronization to a master clock.



Caution

When HW Timestamping is activated, the EthTSyn uses a hardware timer / counter for determination of frame ingress and egress timestamps. This hardware timer / counter module is provided as special PTP (see [8]) feature of some Ethernet controller devices. These additional Ethernet frame timing features are accessed by the EthTSyn over an API extension of the EthIf.

The EthTSyn module offers the functionality to:

- > Provide global network time as master clock role
- > Provide global network time to the StbM for synchronization as slave clock role
- > Use of Hardware or Software for Timestamping

2.1 Architecture Overview

The following figure shows where the EthTSyn is located in the AUTOSAR architecture.

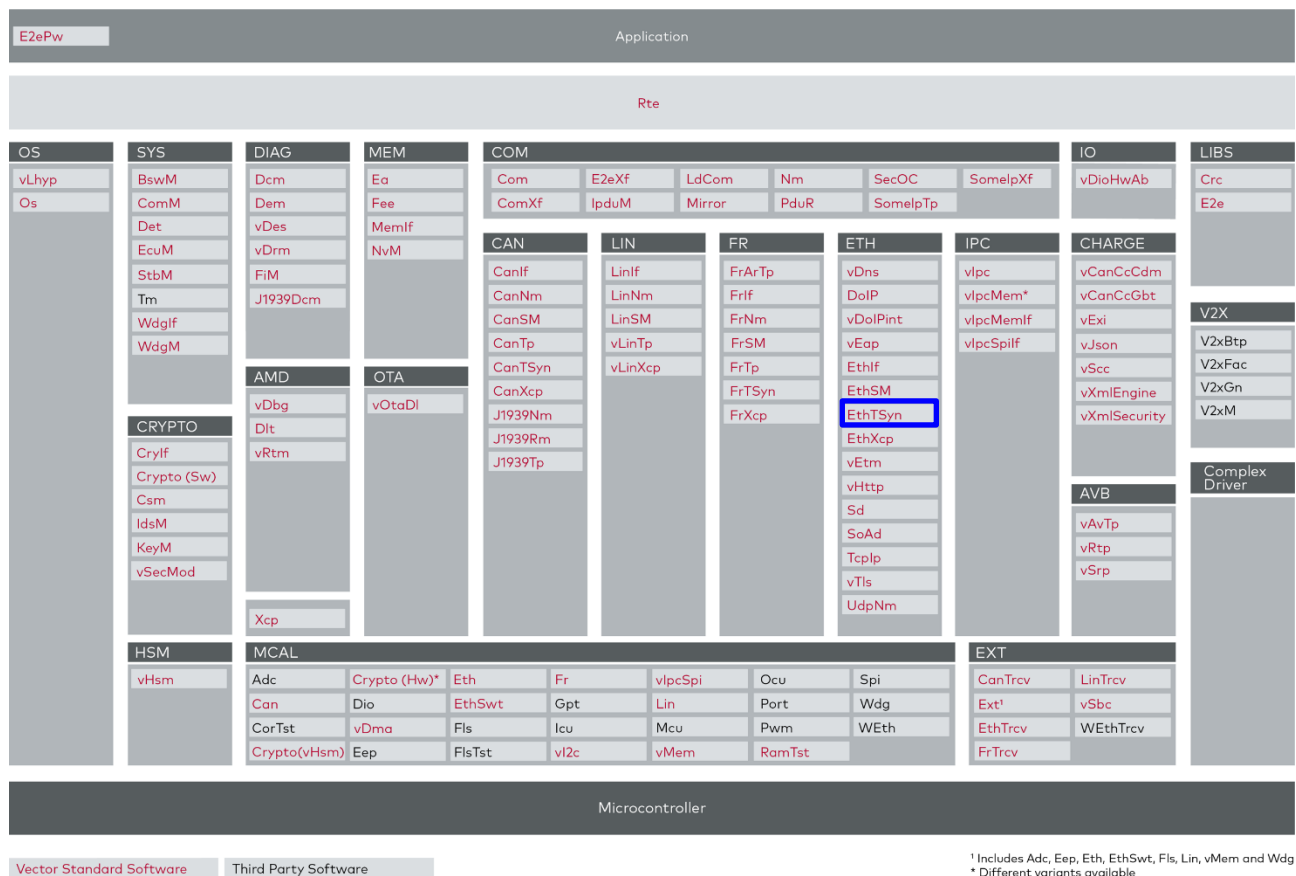


Figure 2-1 AUTOSAR 4.3 Architecture Overview

Figure 2-2 shows the interfaces to adjacent modules of the EthTSyn. These interfaces are described in chapter 5.

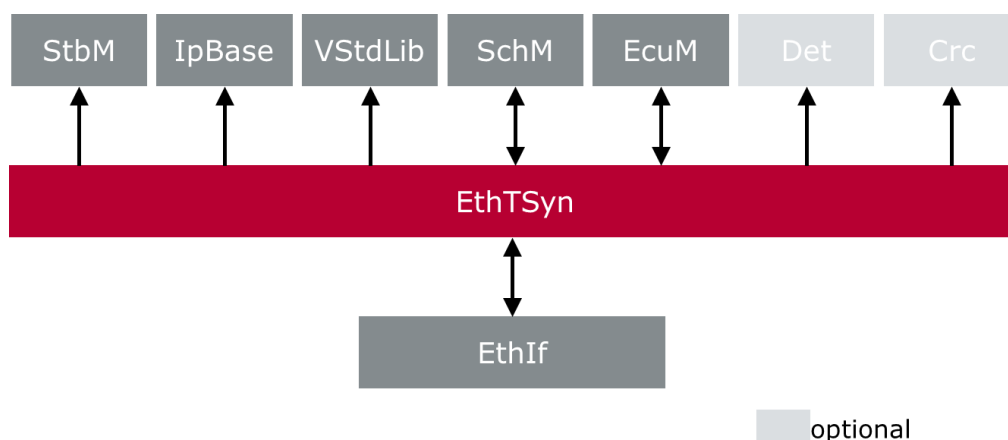


Figure 2-2 Interface to adjacent modules of EthTSyn

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the EthTSyn.

3.1.1 IEEE 802.1AS-2011 Conformance

The following features specified in [7] “IEEE 802.1AS-2011: Timing and Synchronization for Time-Sensitive Applications in Bridged Local Networks” are supported:

Supported features
Clock Master Role
Clock Slave Role

Table 3-1 Feature conformance to IEEE 802.1AS-2011

3.1.2 General Limitations

The EthTSyn can only be used in conjunction with:

- > MICROSAR EthIf
- > MICROSAR StbM

In addition, the following general limitations apply to the EthTSyn.

Not supported features
Best Master Clock Algorithm (BMCA)

Table 3-2 General Limitations

3.1.3 IEEE 802.1AS-2011 Deviations

The following deviations of [7] “IEEE 802.1AS-2011: Timing and Synchronization for Time-Sensitive Applications in Bridged Local Networks” apply to the EthTSyn.

Section in Document	Limitation
6.3.3.4 Data Types	- TimeInterval Format is not supported - ScaledNs Format is not supported - UScaledNs Format is not supported - ExtendedTimestamp Format is not supported
8.2.2 Timescale Epoch	gPTP domain epoch is not supported
8.2.3 Timescale UTC Offset	UTC Offset is not supported
8.6 Time-aware system characterization	No Time-aware system characterization is supported
11.4 Message Format	Header Correction Field: > Only supported in FollowUp messages > Fractional nanoseconds are not supported and ignored on reception

Section in Document	Limitation
	> Value is interpreted as unsigned integer 64 (in accordance with [1])
11.2 State machines	Cyclic transmission of Announce Messages is optional in clock master role
11.2.16 MDPdelay-Req state machine	The loss of allowedLostResponses number of responses is tolerated. The standard allows allowedLostResponses + 1 missing responses before setting AsCapable = false.

Table 3-3 Deviations to see IEEE 802.1AS-2011: Timing and Synchronization for Time-Sensitive Applications in Bridged Local Networks

3.1.4 AUTOSAR 4.4.0 Deviations

There are no deviations to [1] “AUTOSAR_SWS_TimeSyncOverEthernet.pdf”.

3.1.5 Switch management limitations

When switch management mode is used, the following limitations apply.

3.1.5.1 Message limitations

The Time-Aware-Bridge only supports time synchronization and Peer-Delay measurement. This belongs to the gPTP message groups:

- > Sync/FollowUp
- > Pdelay_Req/Pdelay_Resp/Pdelay_Resp_FollowUp

3.1.5.2 AUTOSAR message format limitations

- > A Transparent Clock does not update the CRC of forwarded FollowUp messages
- > A Transparent Clock does not modify, add or remove AUTOSAR Sub-Tlvs
- > A bridge slave port always ignores the CRC of a received FollowUp message. Hence, FollowUp messages including a Sub-Tlv with invalid CRC might be forwarded to the end station and, in case of a Transparent Clock, on all bridge master ports.

3.1.5.3 Slave port limitations

- > In a bridge configuration, Announce can't be used for EndStation slave ports

3.1.5.4 Master port limitations

- > Immediate time synchronization is only applicable when acting as time master or as Boundary Clock

3.1.6 Time-Aware-Bridge (Switch Management)

This feature can be turned on when an EthSwt is included in the configuration by using the configuration parameter `EthTSynEnableSwitchManagement`. The EthTSyn module will then manage all gPTP traffic of the EthSwt. If the Host CPU itself should take Part in the time synchronization, an extra EthTSyn-Port (End-Station Port) for the Host CPU must be configured. The following constraints apply for the configuration of the End-Station Port:

- > If a Switch-Slave Port is configured, the End-Station Port only can be a Slave Port as well.
- > When only Master-Ports are configured for the Switch, an End-Station Master-Port has to be present as well since the Switch itself cannot act as Time Master. In this case, all Switch Master-Ports will inherit the configuration parameters of the End-Station Master-Port.

Figure 3-1 provides an overview of the general structure when an EthSwt is used. The EthTSyn module is running on the Host CPU. The EthSwt is connected with the Host CPU e.g. via MII using the determined management Port of the EthSwt.

**Note**

The EthSwt must run in the so called 'Managed Mode' such that all PTP traffic is trapped and forwarded to the Host CPU only. The normal forwarding rules do not apply to the PTP traffic.

Figure 3-2 provides an overview of the sequence for PTP frames received on an EthSwt port. Figure 3-3 provides an overview of the sequence applying to PTP frames transmitted by the EthTSyn on an EthSwt port.

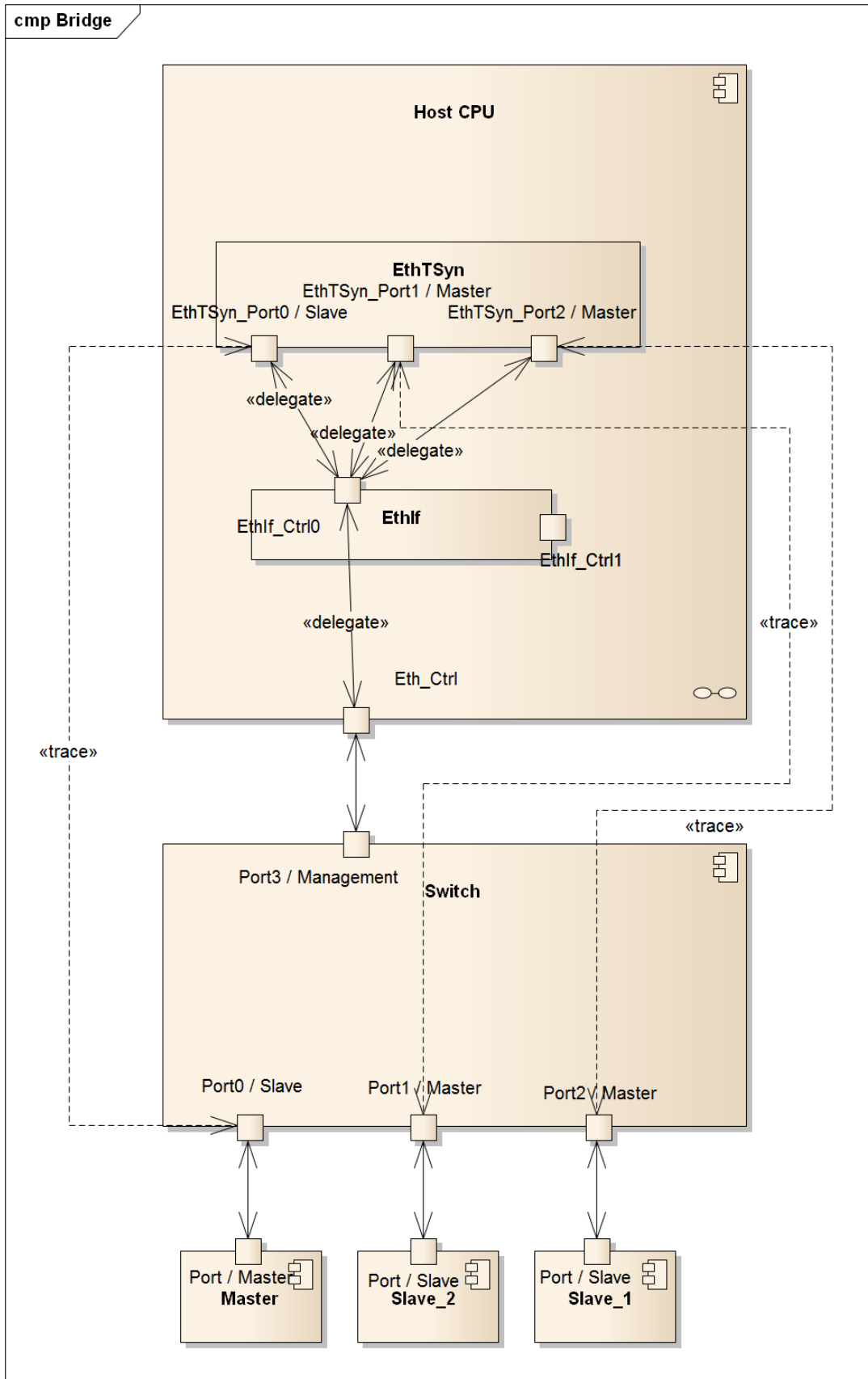


Figure 3-1 Bridge Overview

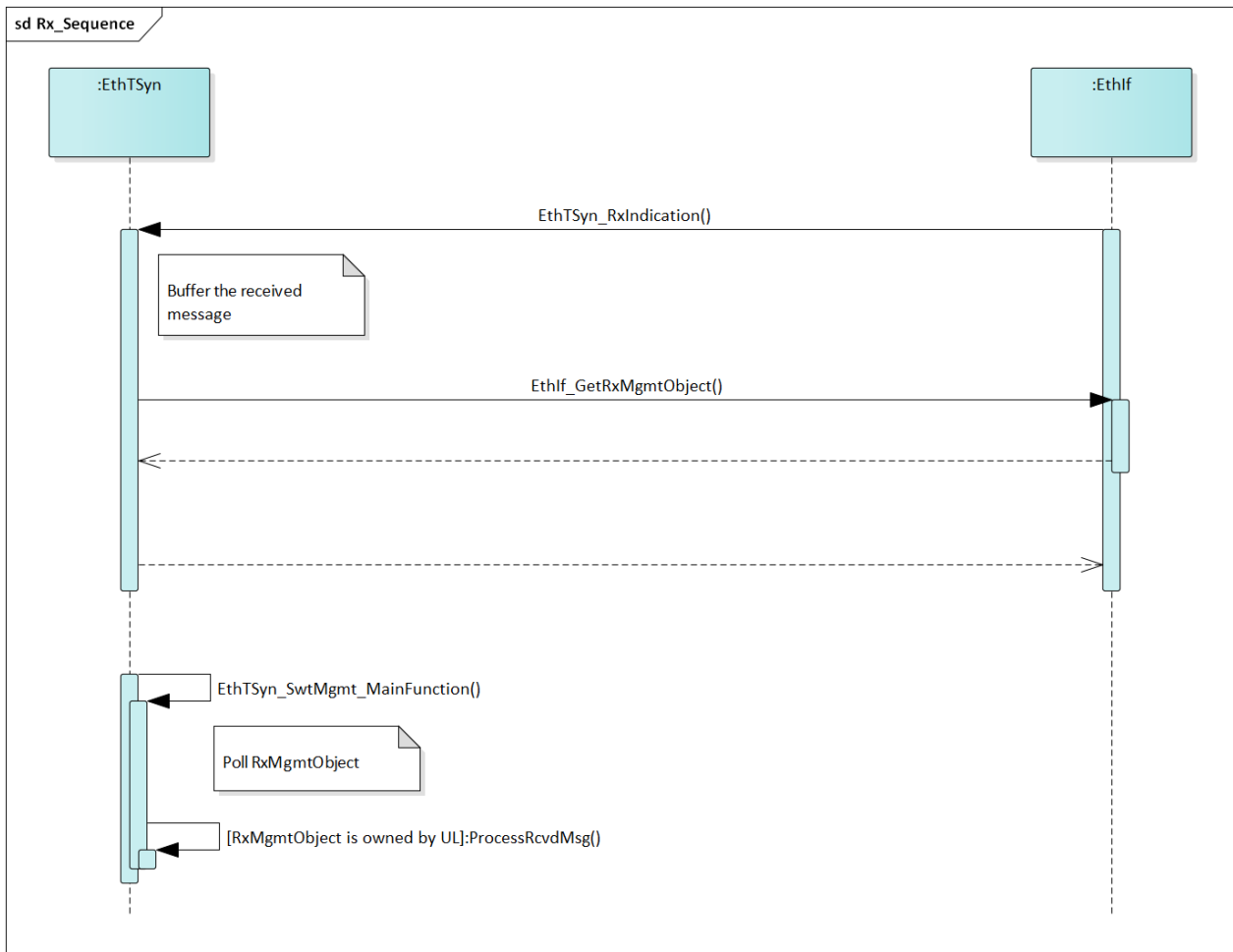


Figure 3-2 Sequence diagram of the switch management reception path

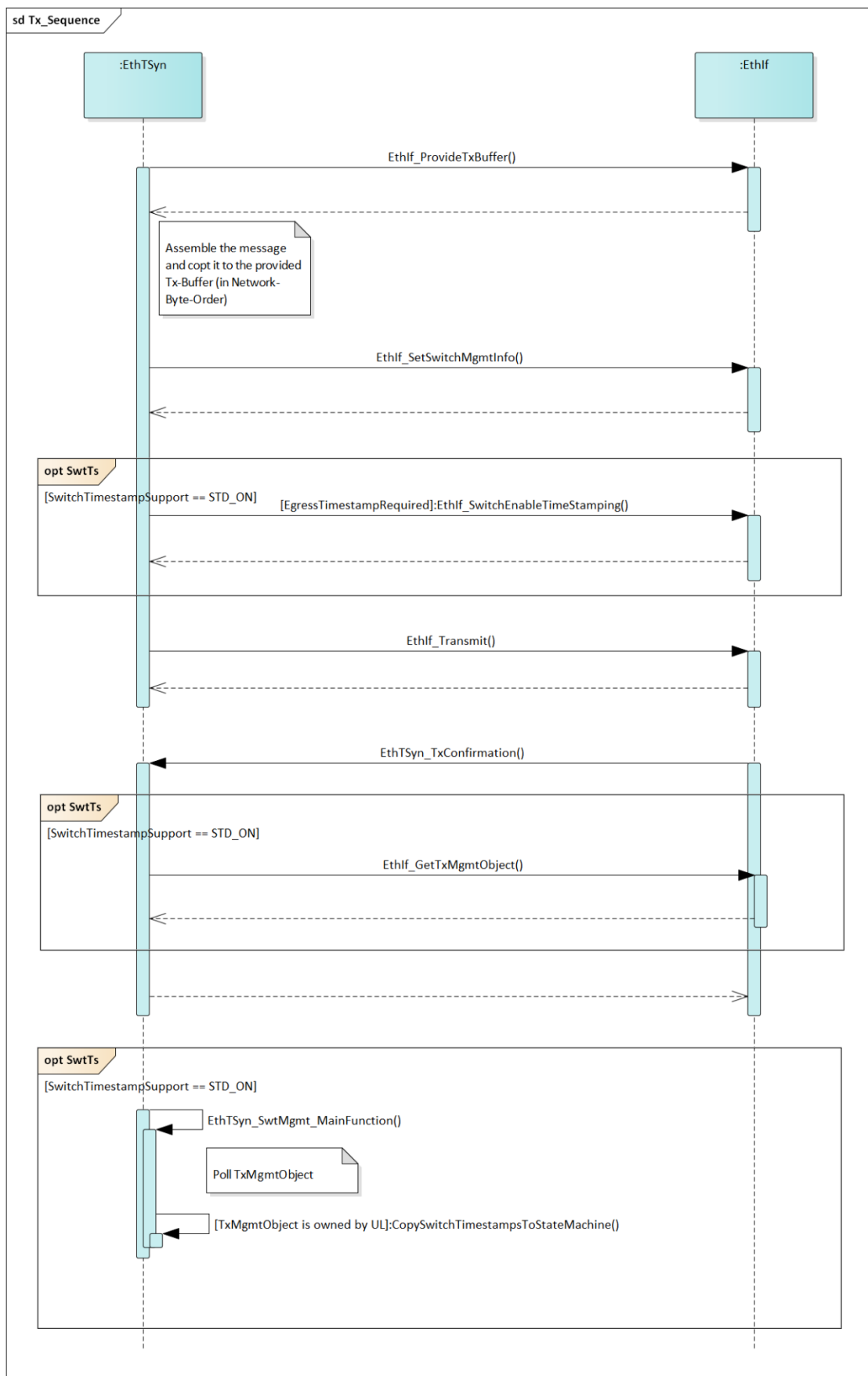


Figure 3-3 Sequence diagram of the switch management transmission path

3.1.6.1 Switch Timestamping

Whether the message timestamps from the Ethernet Switch or the Host CPU are used can be decided by the configuration parameter `EthTSynSwitchMgmtSwitchTimestampSupport`.

When Switch Timestamp Support is enabled in the EthTSyn, the timestamps must be provided by the Ethernet Switch using the corresponding Switch management object.

When Switch Timestamp Support is disabled in the EthTSyn, all necessary timestamps are taken when the corresponding frames are received/transmitted by the Host CPU. Therefore, the switch residence time cannot be computed with disabled Switch timestamp support.

3.1.6.2 Switch and Host as one Time-Aware System

When the Ethernet Switch and the Host controller are considered as one time-aware-system (`EthTSynSwitchMgmtSwitchAndHostAsOneTimeAwareSystem`), the EthTSyn expects that the time of the Ethernet Switch and Host are equal. Therefore, the timestamps provided by the Ethernet Switch can be used as local time.



Caution

The synchronization of the Ethernet Switch and the Host (i.e. Ethernet Controller of the Host) must be ensured. This is not done by the EthTSyn.

3.1.6.3 Time-Aware-Bridge behavior

The behavior of the Time-Aware-Bridge depends on the selected bridge operation mode (`EthTSynSwitchMgmtBridgeMode`). The modes Transparent Clock (3.1.6.4) and Boundary Clock (3.1.6.5) are supported. Both modes are described in more details in the corresponding chapters. Table 3-4 provides a quick comparison of the two operation modes. In addition, chapter 3.1.6.6 describes the behavior when only master ports are configured.

Bridge Mode	Transparent Clock	Boundary Clock
Functionality		
Sync message reception	Received and forwarded	Received and terminated
FollowUp message reception	Received and forwarded	Received and terminated
Sync/FollowUp message transmission	Triggered by reception of the corresponding message or cyclically in case of a sync timeout	Triggered cyclically when <code>GLOBAL_TIME_BASE_BIT</code> of the corresponding time domain is set
Switch residence time	Added to the <code>FollowUpCorretionField</code>	Added to the <code>FollowUpPreciseOriginTimestamp</code>

Sync-Timeout	Optional. When enabled, a sync receive timeout is observed. Upon occurrence of such a timeout, cyclic transmission of Sync/FollowUp messages is started	-
SourcePortIdentity modification	Optional	Mandatory
Pdelay measurement	Configurable for each switch port individually Pdelay measured on the switch slave port is used for the optional synchronization of the Host and added to the FollowUpCorrectionField of a forwarded FollowUp message	Configurable for each switch port individually Pdelay measured on the switch slave port is used for synchronization of the Host only
Acting as Time Master (only master port configured)	Sync/FollowUp messages are cyclically transmitted when the GLOBAL_TIME_BASE_BIT of the corresponding time domain is set. The residence time of the Switch is added to the FollowUpCorrectionField.	Sync/FollowUp messages are cyclically transmitted when the GLOBAL_TIME_BASE_BIT of the corresponding time domain is set. The residence time of the Switch is added to the FollowUpPreciseOriginTimestamp while the FollowUpCorrectionField is set to zero.

Table 3-4 Comparison of Transparent Clock and Boundary Clock

**Note**

The GLOBAL_TIME_BASE_BIT is maintained by the StbM. For more details please refer to [5] AUTOSAR_SWS_SynchronizedTimeBaseManager.pdf

3.1.6.4 Time-Aware-Bridge (Transparent Clock)

In this section, the behavior of a ‘real’ bridge (i.e. one slave port and at least one master port are configured) as transparent clock is described. For more detailed information about a master only configuration, please refer to 3.1.6.6.

3.1.6.4.1 Sync and FollowUp message reception

When a valid Sync message is received on the switch slave port, it is forwarded on each master port of the bridge without waiting for the corresponding FollowUp message. In addition, the Sync message is forwarded to the end station port, if configured.

When a valid FollowUp message is received on the bridge slave port, it is forwarded on each master port if the corresponding Sync message was already transmitted on the port. In addition, the FollowUp message is forwarded to the end station port, if configured.

3.1.6.4.2 Sync and FollowUp message transmission

The Sync and FollowUp message transmission on a bridge master port is either triggered by the reception of the corresponding message (i.e. message is forwarded, see 3.1.6.4.1) or cyclically in case of a sync timeout (see 3.1.6.4.3).

Upon forwarding of a **Sync** or **FollowUp** message, the source port identity might be modified (see 3.1.6.4.4). In addition, the current valid Pdelay of the bridge slave port and the switch residence time of the forwarded **Sync** message are added to the correction field of the forwarded **FollowUp** message.

In case of cyclic message transmission due to a sync timeout, the precise origin timestamp carried in the **FollowUp** message will remain the same as received with the last valid **FollowUp** message. The correction field of the **FollowUp** message will be updated such that the current valid Pdelay of the bridge slave port and the time passed between reception and transmission of the corresponding **Sync** message are included.

3.1.6.4.3 Sync timeout handling

A transparent clock can observe a sync timeout, if `EthTSynSwitchMgmtGmCapable` is enabled and the corresponding `EthTSynSwitchMgmtGlobalTimeSyncTimeout` of the time domain is configured. In case a sync timeout is detected, the transparent clock will start cyclic **Sync** and **FollowUp** message transmission (see 3.1.6.4.2).



Note

The sync timeout observation is started after reception of the first valid **Sync** and **FollowUp** message pair. In case a valid **Sync** is received but no corresponding valid **FollowUp**, no sync timeout is detected.

3.1.6.4.4 Source port identity modification

It is decided by configuration (`EthTSynSwitchMgmtKeepSourcePortIdentity`) if the source port identity of forwarded **Sync** and **FollowUp** messages is modified by the bridge or not:

1. `EthTSynSwitchMgmtKeepSourcePortIdentity` is set to 'false': The source port identity of each forwarded **Sync** and **FollowUp** message will be set to the source port identity of the bridge master port the message is transmitted on
2. `EthTSynSwitchMgmtKeepSourcePortIdentity` is set to 'true': The source port identity will not be modified by the bridge

3.1.6.5 Time-Aware-Bridge (Boundary Clock)

In this section, the behavior of a 'real' bridge (i.e. one slave port and at least one master port are configured) as boundary clock is described. For more detailed information about a master only configuration, please refer to 3.1.6.6. For a boundary clock, the existence of an end station port is mandatory.

3.1.6.5.1 Sync and FollowUp message reception

When a valid **Sync** message is received on the switch slave port, it is forwarded to the end station port and terminated.

When a valid **FollowUp** message is received on the bridge slave port, it is forwarded to the end station port and terminated.

3.1.6.5.2 Sync and FollowUp message transmission

The **Sync** and **FollowUp** message transmission on a bridge master port is triggered cyclically when the `GLOBAL_TIME_BASE_BIT` of the corresponding time domain is set. A new precise origin timestamp is generated on the Host (which includes the switch residence time) for each transmitted **FollowUp** message while the correction field is set to zero.

3.1.6.5.3 Sync timeout handling

A boundary clock always transmits **Sync** and **FollowUp** messages cyclically. Therefore, no sync timeout observation is required.

3.1.6.5.4 Source port identity modification

A boundary clock does not forward received **Sync** and **FollowUp** messages. Upon generation of **Sync** and **FollowUp** messages, the source port identity will always be set according to the bridge master port the message is transmitted on. The configuration option `EthTSynSwitchMgmtKeepSourcePortIdentity` does not influence this behavior.

3.1.6.6 Bridge as Grand Master

In a bridge configuration, where only master ports are configured for a time domain, the bridge is acting as grand master. In this case, **Sync** and **FollowUp** messages will be transmitted on each Master-Port in the configured cycle. Depending on the selected bridge mode (`EthTSynSwitchMgmtBridgeMode`), the residence time of the Ethernet Switch is either used as *FollowUpCorrectionField* (Transparent Clock) or added to the *FollowUpPreciseOriginTimestamp* (Boundary Clock).

3.1.6.7 Pdelay

In a Bridge configuration, the Pdelay mechanism can be configured for each Bridge-Port individually. For a detailed description see chapter 3.1.9.

3.1.7 Clock Master Role

In clock master role the EthTSyn cyclically transmits **Sync** and **FollowUp** from the Ethernet controller (Eth) that is referenced by the EthTSyn GlobalTimeDomain, after the `GLOBAL_TIME_BASE_BIT` is set. The **FollowUp** message contains the *PreciseOriginTimestamp* (egress timestamp of previous **Sync** message) that is generated either by Hardware or Software at transmission of the **Sync** message.

A more detailed description of how the egress timestamp is generated can be found in [1] AUTOSAR_SWS_TimeSyncOverEthernet.pdf, Chapter 9.2.

If enabled, **Announce** messages providing information about the local clock properties are transmitted cyclically with the same cycle time as **Sync** messages. This additional information is used for evaluation of BMCA (Best Master Clock Algorithm) for best master clock determination that is not supported by EthTSyn.

3.1.7.1 Immediate time synchronization

In addition to the regular cyclic time propagation a time master can be configured to propagate its time immediately on changes. This is called immediate time synchronization. An update of the time is detected via the StbM. After an immediate time synchronization event, the time master will wait for the configured cyclic message resume time before transmitting the first regular cyclic `Sync` and `FollowUp` messages again.



Note

In case the cyclic message resume time is too short (e.g. 0s), the time master might not transmit the `FollowUp` message belonging to the immediate `Sync` because a new `Sync` cycle is started before transmission of the `FollowUp` message took place.

3.1.8 Clock Slave Role

In clock slave mode no transmission of `Sync`, `FollowUp` and `Announce` messages is performed. The clock slave receives the `Sync` message from master and stores the ingress timestamp that is either generated by HW or SW. After receiving the `FollowUp` message, the *PreciseOriginTimestamp* of the `FollowUp` message is corrected by the *CorrectionField* of the `FollowUp` message, the path delay time on the link and the time passed between the reception of the `Sync` and `FollowUp` message (by using the ingress timestamp of the `Sync` message). This corrected *PreciseOriginTimestamp* is forwarded to the StbM by using `StbM_BusSetGlobalTime()`.

To determine the path delay of message transmission on this link a path delay measurement (see chapter 3.1.9) is performed.

A more detailed description can be found in [1] AUTOSAR_SWS_TimeSyncOverEthernet.pdf, Chapter 9.3.

3.1.8.1 Announce

According to [7] IEEE 1588-2008: IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems each Time-Master transmits cyclic `Announce` messages. The `Announce` messages are holding information about the Time-Master and the path of messages traversing the network. The Time-Slave is using this information to verify if received `Sync/FollowUp` messages are valid. However, in an AUTOSAR environment no such `Announce` messages will be transmitted.

With the parameter `EthTSynEnableAnnounce` it can be decided if the information of the `Announce` messages is used by the Time-Slave or not.

- > `EthTSynEnableAnnounce` is set to 'false': The information of the `Announce` messages will not be used by the Time-Slave and `Sync/FollowUp` messages with any `SourcePortIdentity` will be accepted (unless the Source Port Identity Check described in 3.1.8.2 is used).
- > `EthTSynEnableAnnounce` is set to 'true': The reception of an `Announce` message by the Slave-Port is mandatory before synchronization is possible. The Slave-Port uses

the PortIdentity with the Announce message to verify the Source Port Identity of received Sync/FollowUp messages. Furthermore the GmPresent flag is set to 'true' after the reception of a valid Announce.

3.1.8.2 Source Port Identity Check

With enabled source port identity check it is possible to configure a master port identity (consisting of a MAC-Address and a port number). Only Sync and FollowUp messages with the configured SourcePortIdentity will be accepted and processed by the slave. All Sync/FollowUp messages with any other SourcePortIdentity will be discarded.

3.1.9 Path Delay (Pdelay) Measurement

In each clock role (master and slave) a cyclic path delay measurement can be performed by transmitting PdelayReq messages. The neighbor node replies by transmitting PdelayResp and PdelayRespFollowUp messages.

When the path delay initiator transmits a PdelayReq message, the egress timestamp is stored. The path delay responder transmits a PdelayResp message in return that contains the ingress timestamp of the PdelayReq message. In addition, the responder transmits a PdelayRespFollowUp message that contains the egress timestamp of the PdelayResp message. Figure 3-4 shows the sequence diagram of the path delay measurement.

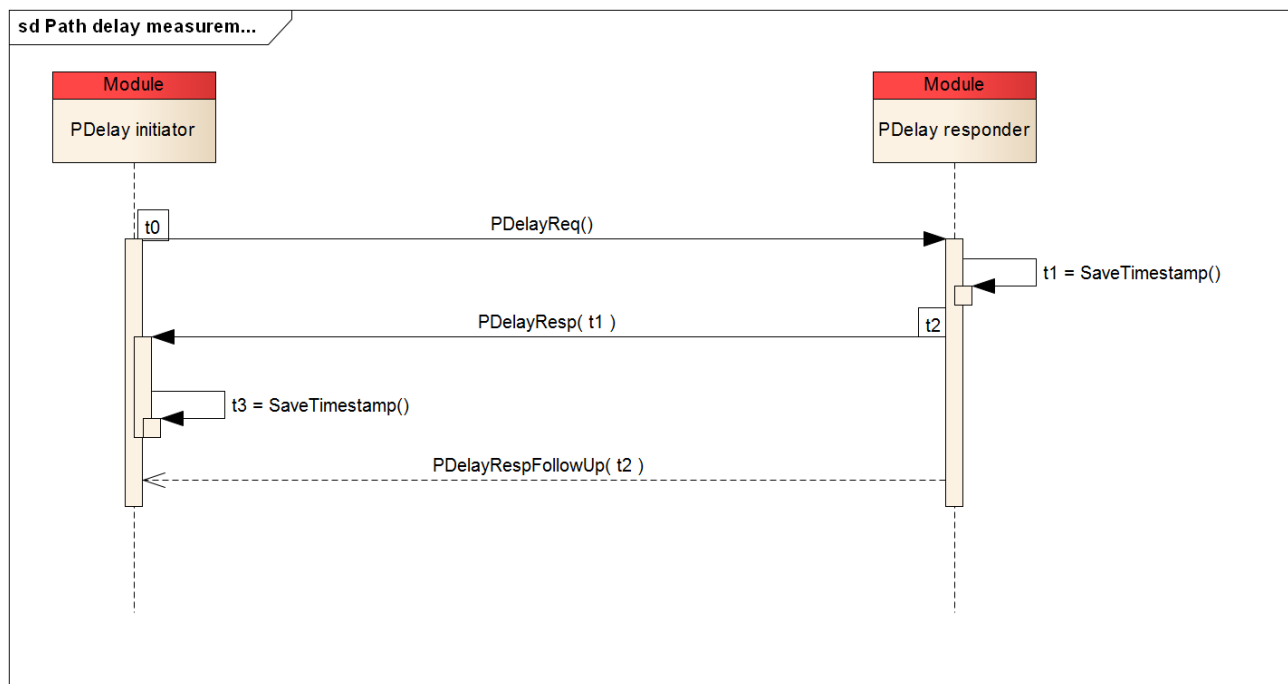


Figure 3-4 Sequence diagram of path delay measurement

3.1.9.1 Flexible Pdelay configuration

The Pdelay configuration options shown in Table 3-5 and Table 3-6 (only relevant if Pdelay Initiator is enabled) can be configured for each Port individually.

Configuration Option	Description
Global Time Tx Pdelay Req Period	The transmission period for <code>PdelayReq</code> messages. A value of 0 disables the Pdelay initiator functionality
Pdelay Responder	Enable or Disable the Pdelay responder functionality. A Pdelay responder will answer to received <code>PdelayReq</code> messages with the corresponding <code>PdelayResp</code> and <code>PdelayRespFollowUp</code> messages.
Use Static Pdelay	Enable or Disable the use of a static configured value for Pdelay. When this option is enabled, the value configured for 'Initial Pdelay' will always be used as Pdelay. No Pdelay calculation will be performed even if the Pdelay Initiator is enabled.
Initial Pdelay	The value initially used for Pdelay (in nanoseconds) before a valid Pdelay was calculated by the Pdelay mechanism. If the option 'Use Static Pdelay' is enabled, this parameter defines the static value used for Pdelay.
Always As-Capable	See 3.1.10

Table 3-5 General Pdelay configuration options

Configuration Option	Description
Pdelay Average Weight	Represents the weight factor (w) for the Pdelay calculation: $PDelay_{New} = \frac{((w - 1) * PDelay_{Old}) + PDelay_{Current}}{w}$
Pdelay Neighbor Delay Threshold	The maximum valid value for Pdelay. If the threshold is exceed, the last valid Pdelay will still be used.
Pdelay Req Allowed Lost Responses	The allowed number of lost responses for a <code>PdelayReq</code> message. If this count is exceeded, 'AsCapable' of the Port is set to false.

Table 3-6 Pdelay initiator configuration options



Caution

If the Pdelay Initiator functionality is disabled, the configuration option 'Always AsCapable' must be enabled.

**Note**

If the configuration options for both Pdelay Initiator and Use Static Pdelay are disabled, no Pdelay is used for the Port and therefore the value for Initial Pdelay must be 0.

3.1.10 AlwaysAsCapable

When AlwaysAsCapable is set to true, AsCapable is set to true without checking for a gPTP counterpart via Pdelay-Mechanism. Hence for a master port the transmission of Sync and Follow_Up messages is possible without Pdelay measurement. For a slave port, synchronization is possible without a valid Pdelay.

3.1.11 Acting as Time-Master and Time-Slave in parallel

Each physical Port can act as Time-Master and Time-Slave in parallel but can only have one role within the same TimeDomain. This means a port can act as Time-Slave in one TimeDomain and act as Time-Master in another TimeDomain.

3.1.12 Debounce time

In order to avoid traffic bursts on an Ethernet link, the EthTSyn is capable to use a debounce time mechanism. The debounce time describes the minimal time interval between the transmission of two EthTSyn messages on one Ethernet link. It is therefore independent of the TimeDomain. Although the debounce time is configurable for each EthTSyn port individually, it must be ensured that the debounce time is equal for each EthTSyn port using the same underlying HW-Port. Usage of the debounce time mechanism can be de-/activated by configuration.

The following three message groups describe the transmission priority if more than one message is waiting for transmission when the debounce time expires:

1. Sync, Follow_Up, Announce
2. Pdelay_Req
3. Pdelay_Resp, Pdelay_Resp_Fup

The groups are listed in order of descending priority. Sync messages, for example, are transmitted with a higher priority than Pdelay_Req messages. The priority of messages within the same message group is determined by the TimeDomain Id where lower Id means higher priority. An Announce message with TimeDomain Id = 3, for example, has a higher priority than a Sync message with Id = 5.

Deviations from this priority can occur if the transmission of one message fails. In this case, the message with the second highest priority will be transmitted and the debounce timer is restarted. A new attempt for transmitting the first message is only possible after the debounce time is expired again.

As consequence of the prioritization, the debounce time must be chosen carefully with the number of different messages and their cyclic transmission times in mind. Otherwise, the

cyclic transmission can be prolonged or lower priority messages will never be transmitted. The following examples illustrate possible problems:

- > In case that Sync and Follow_Up messages of one time domain are transmitted with a cyclic transmission time of 100 ms, the debounce time must not be more than 50 ms. If it is for example 70 ms, two consecutive Sync messages are transmitted with a time difference of 140 ms which effectively increases the cyclic transmission time by 40 ms.
- > In case that Sync and Follow_Up messages of five time domain are transmitted with a cyclic transmission time of 100 ms each, the debounce time must not be more than 10 ms. Otherwise, the messages of the time domain with highest Id will never be transmitted because there is always a message with higher priority ready for transmission.
- > A similar problem can occur for messages of different groups. In case that Sync and Follow_Up messages of one time domain and Pdelay_Req messages are transmitted with a cyclic transmission time of 100 ms each, the debounce time must not be more than 33 ms. Otherwise, the Pdelay_Req message will never be transmitted.
- > If switch management is enabled and the bridge mode is transparent clock, received Sync and Follow_Up messages are forwarded to all master port. Consequently, the maximum possible debounce time also depends on the cyclic transmission time of the global time grandmaster.

3.1.13 Time synchronization of ethernet switches

For some use cases (e.g. TSN) it might be necessary to synchronize the time of an ethernet switch. The EthTSyn offers the possibility to handle the time synchronization of one or multiple ethernet switches. For information about the configuration for switch time synchronization refer to chapter 6.2.



Caution

A version of 11.00.00 or greater of the MICROSAR EthIf is required to use this Feature.

3.1.13.1 Cascaded switches

For the synchronization of cascaded switches, a Master-Slave concept is used. The switch which is directly connected to the grand master must be the Master-Switch for the cascade. All performed corrections are only calculated for the Master-Switch and applied to the Master-Switch and all Slave-Switches of the cascade.

A separate synchronization of all cascaded switches is not supported.



Caution

Limitations for cascaded switches:

For the synchronization of cascaded switches, it is mandatory that the PTP Clock of all involved switches is driven by the same oscillator or that the Master-Switch is providing the PTP Clock for all Slave-Switches.

3.1.13.2 Notification about changes in the switch synchronization state

For each switch cascade which is synchronized by the EthTSyn, one user can register a notification for changes in the synchronization state of the switch cascade. An overview about the reported synchronization states and their meaning can be found in Table 3-7.

Reported switch sync state	Description
ETHTSYN_SYNC	The time of the switch cascade is synchronized to the time master, i.e. the computed switch time offset with the last sync event does not exceed the configured sync precision limit.
ETHTSYN_UNSYNC	The time of the switch cascade is not synchronized to the time master, i.e. the computed switch time offset exceeded the configured sync precision limit for at least 'max out of sync count' consecutive sync events.
ETHTSYN_UNCERTAIN	It is unclear whether the switch time is synchronized to the time master or not. This happens when: ▶ Computation of the time offset failed ▶ Applying the time correction to at least one of the switches failed ▶ Synchronization event timeout was detected

Table 3-7 Reported switch sync states

3.1.13.3 Rate regulator time synchronization mechanism

The rate regulator mechanism is used for time synchronization of Ethernet switches. The basic idea is to synchronize without performing time leaps for offset correction. Hence, only the rate ratio (i.e. clock frequency) is adapted in such a way that the offset is zero after a certain number of synchronization events.

The adapted rate ratio is determined by a configured number (N) of parallel rate measurements which all start and end with a synchronization event. This process is illustrated in Figure 3-5 for $N = 3$ rate measurements. The horizontal axis describes the progression of time. Blue arrows represent points of time when synchronization events occur. Red, yellow and green lines are the three rate measurements. Dots on these lines

mark the end and beginning of the respective measurement. The measurements begin at successive synchronization events and the first measurement does not end until the last measurement started. Consequently, N is also the duration of each rate measurement when this duration is measured in number of synchronization cycles. The red, yellow and green vertical arrows indicate which measurement was used to calculate the new rate ratio (RR) at a given synchronization event.

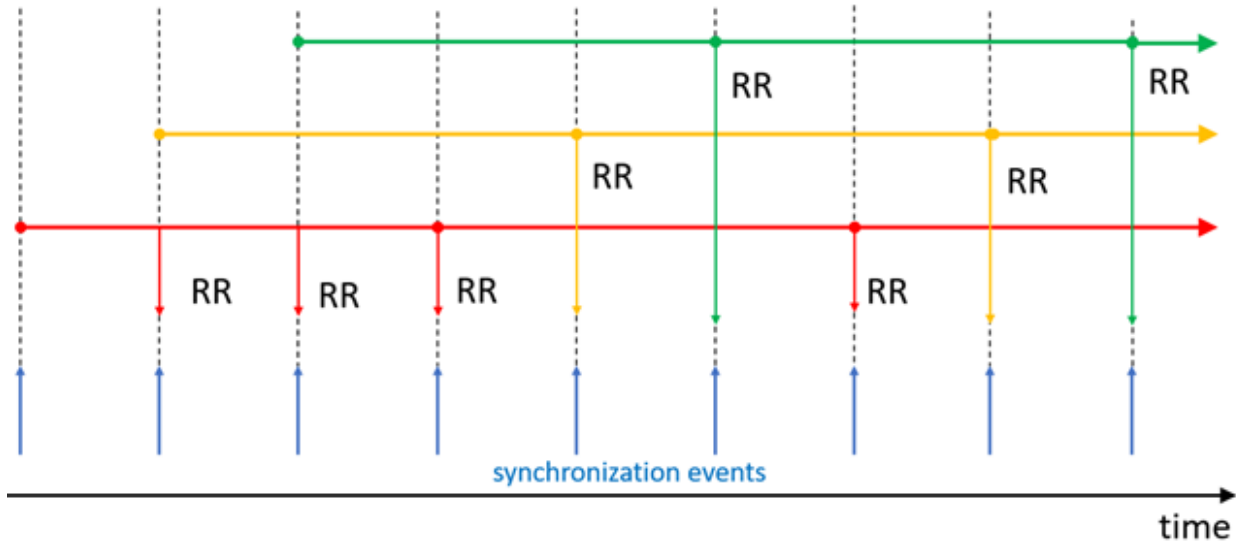


Figure 3-5 Schematic representation of the measurement sequence used by the rate regulator time synchronization mechanism

The rate ratio itself consists of two components: the grand master rate ratio (RR^{GM}) and the offset correction rate ratio (RR^{Off}) which are calculated according to the following equation:

$$RR = RR^{GM} + RR^{Off} = \frac{MTD}{STD} + \frac{N}{M} \frac{Off}{STD}$$

MTD and STD are the differences of the master time and switch time, respectively, between two successive synchronization events. If the duration of the rate measurements is longer than one synchronization cycle ($N > 1$), the accumulated value of MTD and STD over all synchronization cycles of the current measurement is used for the calculation.

The grand master rate ratio corrects the difference of the clock frequencies between master and slave whereas the offset correction rate ratio adjusts the frequency in a way that the offset vanishes after a configured number (M) of further synchronization cycles. If the mechanism is configured to correct the offset during one synchronization cycle ($M = 1$), it can lead to an overshoot of the synchronization mechanism and induce a further offset. A higher value of M might increase the duration until synchronization is established but stabilizes the regulation.

Another possible source for errors is a random fluctuation of the duration of each synchronization cycle (e.g. due to jitter). This leads to an error in the calculation of the offset correction rate ratio which can in turn induce a further offset and prevent the system from synchronizing. This effect can be reduced with a higher value of N because the increased duration of each rate measurement decreases the impact of random errors.

There are two configurable options in which the rate regulator uses time leap to correct the offset. They are described in the following subchapters.

3.1.13.3.1 Initial offset correction

After initialization, the clocks of different devices involved in the time synchronization can have significantly different time values. For example, the global time master might receive its clock from an external source (e.g. GPS) while the slave just starts with a time of zero. The time offset between master and slave can therefore be so high that the correction via rate ratio might not be possible or take too much time until synchronization is established.

In order to prevent this, it is possible to enable an offset correction via time leap and grand master rate ratio before synchronization was established the first time.

3.1.13.3.2 Offset jump correction threshold

If the clock of the time master exhibits a time leap during operation, the resulting offset between master and slave can be so high that the correction via rate ratio might not be possible or take too much time until synchronization is established.

In order to prevent this, it is possible to configure an offset jump correction threshold. If the offset exceeds the value of this threshold, it will be corrected by time leap instead of rate regulation.



Caution

It is assumed that the rate ratio (i.e. clock speed ratio) between master and switch is relatively small (~200ppm, depending on the clock source).

In case the offset jump correction is used and the configured threshold is smaller than the offset caused by the rate ratio within one sync cycle, no synchronization can be established.

3.1.14 Time validation

When the time validation feature is enabled for a time domain, the EthTSyn reports the data used for time synchronization (Master/Slave) and Pdelay calculation (PdelayInitiator/PdelayResponder) of the time domain to the StbM by using the corresponding Interfaces of the StbM. For a more detailed description of the feature please refer to [2] AUTOSAR_SWS_TimeSyncOverEthernet.pdf and [6] AUTOSAR_SWS_SynchronizedTimeBaseManager.pdf

**Note**

The EthTSyn mainly follows AUTOSAR version 4.4. However, the time validation was introduced with Version 19-11 and is implemented according to this version with the following deviations:

- > The parameter `EthTSynTimeValidationSupport` in the `EthTSynGeneral` container is not available. The time validation feature is automatically activated depending on the setting within the referenced `StbMSynchronizedTimeBase`
- > It is expected that a `StbMSynchronizedTimeBase` includes an optional container `StbMTimeValidation`. The existence of this container enables the time validation feature.
- > It is expected that the structure `StbM_EthTimeMasterMeasurementType` has a member `correctionField` (`sint64`) (like the `StbM_EthTimeSlaveMeasurementType`)

3.2 Interaction with EthIf and StbM BSW module

The interaction of the EthTSyn with the EthIf and StbM BSW modules is described in [1] [AUTOSAR_SWS_TimeSyncOverEthernet.pdf](#).

**Caution**

It is assumed that the times retrieved via `EthIf_GetCurrentTime()`, `EthIf_GetIngressTimeStamp()`, `EthIf_GetEgressTimeStamp()` and `StbM_GetCurrentVirtualLocalTime()` are monotonously increasing. In addition, times retrieved from the HW via EthIf are transformed into a virtual local time. Therefore, it is expected that these times do not exceed the value range of $(2^{64} - 1)$ nanoseconds (~584 years).

3.3 Initialization

The EthTSyn is initialized by calling the `EthTSyn_InitMemory()` and `EthTSyn_Init()` services with the address of the pre-compile configuration data passed as parameter. The `EthTSyn_InitMemory()` function has to be called before `EthTSyn_Init()` to initialize used memory of the EthTSyn module.

The EthTSyn port configuration is pre-defined by Configurator Pro configuration process.

**Note**

Since EthTSyn is using the EthIf and Eth BSW module they must be initialized before executing the EthTSyn initialization process.

3.4 States

The EthTSyn is operational after initialization. The EthTSyn transmits `PdelayRequest` messages periodically to determine the path delay to a connected EthTSyn device. If no `PdelayResponse` is received, no further actions for synchronization are taken. A list of implemented state machines can be found in Table 3-8. The detailed description of each state machine is presented in IEEE 1588-2008: IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems.

State machine	Task
<code>EthTSyn_Slave_ProcessSmSyncReceive()</code>	Processing of received <code>Sync</code> and <code>FollowUp</code> messages in EthTSyn port slave role.
<code>EthTSyn_Pdelay_ProcSmPdReq()</code>	Periodic transmission of <code>PdelayRequest</code> messages and processing of received <code>PdelayResponse</code> and <code>PdelayResponseFollowUp</code> messages
<code>EthTSyn_Pdelay_ProcSmPdResp()</code>	Processing of received <code>PdelayRequest</code> messages and transmission of <code>PdelayResponse</code> and <code>PdelayResponseFollowUp</code> messages.
<code>EthTSyn_Master_ProcessSmSyncSend()</code>	Periodic transmission of <code>Sync</code> and <code>FollowUp</code> in port master role.
<code>EthTSyn_Slave_ProcessSmAnnounceReceive()</code>	Processing of received <code>Announce</code> messages in EthTSyn port slave role.
<code>EthTSyn_SwtMgmt_ProcessSmSiteSyncSync()</code>	Processing of <code>Sync</code> and <code>FollowUp</code> received on the slave port and to be forwarded on each master port.

Table 3-8 EthTSyn state machines

3.5 Main Functions

The EthTSyn has multiple main functions. Depending on the configuration they exist or not. The following tables list all main functions with information about existence and their purpose.

EthTSyn_MainFunction()		
Existence	Timing	Description
Always	Scheduled according to the period provided by configuration parameter <code>EthTSynMainFunctionPeriod</code> .	Processes all state machines and timers of the EthTSyn.

Table 3-9 EthTSyn_MainFunction() description

EthTSyn_SwtMgmt_MainFunction()		
Existence	Timing	Description
EthTSynEnableSwitchMgmt is enabled	Scheduled according to the period provided by configuration parameter EthTSynSwitchMgmtMainFunctionPeriod.	Processes the switch management Rx/Tx management objects as well as received messages.

Table 3-10 EthTSyn_SwtMgmt_MainFunction() description

3.6 Error Handling

3.6.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [3], if development error reporting is enabled (i.e. pre-compile parameter `ETHTSYN_DEV_ERROR_REPORT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported EthTSyn ID is 164.

The reported service IDs identify the services which are described in chapters 5.2 and 5.4. The following table presents the service IDs and the related services:

Service ID	Service
0x01	EthTSyn_Init()
0x02	EthTSyn_GetVersionInfo()
0x05	EthTSyn_SetTransmissionMode()
0x06	EthTSyn_RxIndication()
0x07	EthTSyn_TxConfirmation()
0x08	EthTSyn_TrcvLinkStateChg()
0x09	EthTSyn_MainFunction()
0x51	EthTSyn_SwtMgmt_MainFunction()

Table 3-11 Service IDs: Development error reporting

The errors reported to DET are described in the following table:

Error Code	
0x20	ETHTSYN_E_NOT_INITIALIZED
0x21	ETHTSYN_E_INIT_FAILED
0x22	ETHTSYN_E_CTRL_IDX
0x23	ETHTSYN_E_PARAM_POINTER
0x24	ETHTSYN_E_PARAM
0x30	ETHTSYN_E_MSG_LENGTH

Error Code	
0x31	ETHTSYN_E_PROTOCOL_VERSION
0x32	ETHTSYN_E_MSG_TYPE
0x33	ETHTSYN_E_FRAME_TYPE
0x34	ETHTSYN_E_DOMAIN_NUMBER
0x35	ETHTSYN_E_TRCV_DOWN
0x36	ETHTSYN_E_NO_PORT_FOUND
0x37	ETHTSYN_E_FUP_TX_LENGTH
0x50	ETHTSYN_SWT_MGMT_E_MSG_BUFFER_OVERFLOW
0x51	ETHTSYN_SWT_MGMT_E_MSG_BUFFER_PAYLOAD_OVERFLOW
0x52	ETHTSYN_SWT_MGMT_E_GET_RX_MGMT_OBJ_FAILED

Table 3-12 Development errors reported to DET

3.6.2 Runtime Error Reporting

By default, runtime error are reported to the DET using the service `Det_ReportRuntimeError()` as specified in see [3], if master-slave conflict detection is enabled (i.e. `ETHTSYN_MASTER_SLAVE_CONFLICT_DETECTION==STD_ON`).

The reported EthTSyn ID is 164.

The following table presents the service IDs and the related services:

Service ID	Service
0x50	EthTSyn_UtilRx_SyncPortRoleCheckWithMasterSlaveConflictDetection()

Table 3-13 Service IDs: Runtime error reporting

The errors reported to DET are described in the following table:

Error Code	
0x01	ETHTSYN_E_TMCONFLICT
0x02	ETHTSYN_E_TSCONFLICT

Table 3-14 Runtime errors reported to DET

3.6.3 Production Code Error Reporting

No production error code reporting to the DEM (see [4]) is supported.

Error Code	Description
none	

Table 3-15 Errors reported to DEM

4 Integration

This chapter gives necessary information for the integration of the MICROSAR EthTSyn into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the EthTSyn contains the files which are described in the chapters 4.1.1 and 4.1.2.

4.1.1 Static Files

File Name	Description
EthTSyn.c	Implementation of the EthTSyn core functionality
EthTSyn.h	API declaration
EthTSyn_Cbk.h	API call-back declaration
EthTSyn_Crc_Int.c	Implementation of the EthTSyn Crc functionality
EthTSyn_Crc_Int.h	Internal macro and function definition to handle Crc calculation and validation
EthTSyn_Int.h	Internal API declaration
EthTSyn_Master_Int.c	Implementation of the EthTSyn Master functionality
EthTSyn_Master_Int.h	Internal API declarations with respect to the Master functionality
EthTSyn_Pdelay_Int.c	Implementation of the EthTSyn Pdelay functionality
EthTSyn_Pdelay_Int.h	Internal API declarations with respect to the Pdelay functionality
EthTSyn_Slave_Int.c	Implementation of the EthTSyn Slave functionality
EthTSyn_Slave_Int.h	Internal API declarations with respect to the Slave functionality
EthTSyn_SwtMgmt_Int.c	Implementation of the EthTSyn Switch Management functionality
EthTSyn_SwtMgmt_Int.h	Switch Management internal API declaration
EthTSyn_SwtTimeSync_Int.c	Implementation of the ethernet switch time synchronization functionality
EthTSyn_SwtTimeSync_Int.h	Internal API declarations with respect to the ethernet switch time synchronization functionality
EthTSyn_TimeValidation_Int.c	Implementation of the EthTSyn Time Validation functionality
EthTSyn_TimeValidation_Int.h	Internal API declarations of the Time Validation unit
EthTSyn_Types.h	Type definitions for EthTSyn modules
EthTSyn_Types_Int.h	Internal type definitions for EthTSyn modules
EthTSyn_Util_Int.c	Implementation of utilities used by different sub-modules of the EthTSyn

File Name	Description
EthTSyn_Util_Int.h	Internal API declarations of utilities used by different sub-modules of the EthTSyn

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator Pro.

File Name	Description
EthTSyn_Cfg.h	Generated header file for pre-compile time configuration data
EthTSyn_GeneratedCfgAccess.h	Generated header file with functions/macros for access of the generated data
EthTSyn_GeneratedComplexTypes.h	Generated header file with complex (i.e. structs, etc.) type definitions required for generated data
EthTSyn_GeneratedSimpleTypes.h	Generated header files with basic type definitions required for generated data
EthTSyn_Lcfg.c	Generated pre-compile and link time configuration data
EthTSyn_Lcfg.h	Generated header for pre-compile and link time configuration data

Table 4-2 Generated files

4.2 Critical Sections

The EthTSyn uses the Critical Section implementation of the SchM.

4.2.1 Exclusive Area 0

ETHTSYN_EXCLUSIVE_AREA_0
Purpose: Ensures consistency of global RAM variables. Interfaces: > SchM_Enter_EthTSyn_ETHTSYN_EXCLUSIVE_AREA_0 > SchM_Exit_EthTSyn_ETHTSYN_EXCLUSIVE_AREA_0 Runtime: Long: Depending on the API and call graph. Dependency: > EthTSyn_Master_SyncSend_IsFupTxReady() > EthTSyn_Master_TxAnnounceMsg() > EthTSyn_Master_HandleCyclicTasks()

```
> EthTSyn_Master_SyncSend_TxFup()
> EthTSyn_Master_SyncSend_HndlWaitSyncEgrTsState()
> EthTSyn_Master_PortSyncSend_InitInternal()
> EthTSyn_Master_PortSyncSend_ProcSyncTxConf()
> EthTSyn_Pdelay_TxPdelayReqMsg()
> EthTSyn_Pdelay_TxPdelayRespMsg()
> EthTSyn_Pdelay_TxPdelayRespFupMsg()
> EthTSyn_Pdelay_CheckForPdelayRespTimeout()
> EthTSyn_Pdelay_HandleCyclicTasks()
> EthTSyn_Pdelay_ProcRcvdReqMsg()
> EthTSyn_Pdelay_ProcRcvdRespFupMsg()
> EthTSyn_Slave_ProcessSmSyncReceive()
> EthTSyn_Slave_HandleCyclicTasks()
> EthTSyn_SlaveRx_ProcRcvdSyncMsg()
> EthTSyn_SlaveRx_ProcRcvdAnnounceMsg()
> EthTSyn_SlaveRx_WriteSyncMsgToSyncReceiveSm()
> EthTSyn_SwtMgmt_Rx_FwdSyncMsgToEndStation()
> EthTSyn_SwtMgmt_Rx_ProcRcvdSyncMsg()
> EthTSyn_SwtMgmt_Rx_ProcRcvdFupMsg()
> EthTSyn_SwtMgmt_SiteSyncSync_HndlPendingState()
> EthTSyn_SwtMgmt_SiteSyncSync_HndlSyncRetry()
> EthTSyn_SwtMgmt_SiteSyncSync_HndlFupRetry()
> EthTSyn_SwtMgmt_SiteSyncSync_RcvdSync()
> EthTSyn_SwtMgmt_SiteSyncSync_RcvdFup()
> EthTSyn_SwtMgmt_RxBufHndl_Poll()
> EthTSyn_SwtMgmt_RxBufHndl_Release()
> EthTSyn_SwtMgmt_RxBufHndl_ReleaseAllPending()
> EthTSyn_SwtMgmt_TxMgmtObjHndl_Poll()
> EthTSyn_SwtMgmt_TxMgmtObjHndl_ProcPending()
```

```
> EthTSyn_SwtMgmt_TxMgmtObjHndl_ReleaseAllPending()  
> EthTSyn_SwtTimeSync_HandleSwtTimeSync()  
> EthTSyn_SwtTimeSync_ComputeMasterAndSwitchTimeDiff()  
> EthTSyn_SwtTimeSync_ProvideTimestampsFromSwtSlavePort()
```

Recommendation:

It is recommended to use the functions `SuspendAllInterrupts()` and `ResumeAllInterrupts()` for this critical section.

Table 4-3 Exclusive Area 0

4.2.2 Exclusive Area Get Ts

ETHTSYN_EXCLUSIVE_AREA_GET_TS

Purpose:

- > Ensures consistency of global RAM variables related to ingress and egress timestamping
- > Prevents interrupts between invocation of `EthTSyn_TxConfirmation` respectively `EthTSyn_RxIndication` and getting the egress respectively ingress timestamp to minimize the delay
- > Prevents interrupts between getting the current Ethernet time via `EthIf_GetCurrentTime` and getting the current virtual local time via `StbM_GetCurrentVirtualLocalTime` to minimize the delay

Interfaces:

- > `SchM_Enter_EthTSyn_ETHTSYN_EXCLUSIVE_AREA_GET_TS`
- > `SchM_Exit_EthTSyn_ETHTSYN_EXCLUSIVE_AREA_GET_TS`

Runtime:

Long: This exclusive area covers calls to several sub-functions and to foreign modules (`EthIf` and `StbM`).

Dependency:

- > `EthTSyn_Master_CalcPot()`
- > `EthTSyn_SlaveRx_GetTimeDiffSinceSyncRx()`
- > `EthTSyn_RxInd_IntRxIndication()`
- > `EthTSyn_TxConfirmation()`

Recommendation:

It is recommended to use the functions `SuspendAllInterrupts()` and `ResumeAllInterrupts()` for this critical section, unless the StbM uses the Os as time source.

Table 4-4 Exclusive Area Get Ts



Caution

If the StbM is configured to use OS APIs and the implementation method of the exclusive area `ETHTSYN_EXCLUSIVE_AREA_GET_TS` is configured to `OS_INTERRUPT_BLOCKING` or `ALL_INTERRUPT_BLOCKING`, the OS may report the error `E_OS_DISABLEDINT` notified by the OS `ErrorHook()`. In that case the implementation method of the exclusive area `ETHTSYN_EXCLUSIVE_AREA_GET_TS` inside the RTE / SchM configuration needs to be set to `OS_RESOURCE` or `CUSTOM`.

If `OS_RESOURCE` is selected the ETH ISR(s) need to reference the OS Resource created by the RTE.

If `CUSTOM` is selected the SchM APIs for entering and exiting the exclusive area need to be implemented manually by using an interrupt lock mechanism but without calling OS APIs like `SuspendOsInterrupts()` or `DisableAllInterrupts()`.

Note:

The exclusive area implementation method `CUSTOM` is a MICROSAR RTE extension and might not be available in other RTEs.

4.2.3 Exclusive Area Time Valid

ETHTSYN_EXCLUSIVE_AREA_TIME_VALID

Purpose:

- > Ensures consistency of global RAM variables related to time validation
- > Prevents interrupts between reading/writing time validation data

Interfaces:

- > `SchM_Enter_EthTSyn_ETHTSYN_EXCLUSIVE_AREA_TIME_VALID`
- > `SchM_Exit_EthTSyn_ETHTSYN_EXCLUSIVE_AREA_TIME_VALID`

Runtime:

Medium: This exclusive area covers calls to internal timestamp conversion functions (call depth ≤ 1).

Dependency:

- > `EthTSyn_TimeValid_MasterReportData()`

```
> EthTSyn_TimeValid_MasterStartCycle()
> EthTSyn_TimeValid_MasterProvideFupData()
> EthTSyn_TimeValid_MasterProvideSyncEgressTs()
> EthTSyn_TimeValid_MasterFollowUpSent()
> EthTSyn_TimeValid_SlaveReportData()
> EthTSyn_TimeValid_SlaveStartCycle()
> EthTSyn_TimeValid_SlaveProvideSyncIngressData()
> EthTSyn_TimeValid_SlaveProvideFollowUpData()
> EthTSyn_TimeValid_SlaveProvideCurrPdelay()
> EthTSyn_TimeValid_SlaveBusSetGlobalTimeCalled()
> EthTSyn_TimeValid_PdReqReportData()
> EthTSyn_TimeValid_PdReqStartCycle()
> EthTSyn_TimeValid_PdReqTakeRefTimes()
> EthTSyn_TimeValid_PdReqCopyDataFromSm()
> EthTSyn_TimeValid_PdRespReportData()
> EthTSyn_TimeValid_PdRespStartCycle()
> EthTSyn_TimeValid_PdRespTakeRefTimes()
> EthTSyn_TimeValid_PdRespProvidePdReqData()
> EthTSyn_TimeValid_PdRespProvidePdRespEgressTs()
> EthTSyn_TimeValid_PdRespPdRespFupSent()
```

Recommendation:

It is recommended to use the functions `SuspendAllInterrupts()` and `ResumeAllInterrupts()` for this critical section.

Table 4-5 Exclusive Area Time Valid

4.3 Main Functions

The EthTSyn expects that its own main functions do not interrupt each other. Furthermore, it expects that its own main functions do not interrupt the main functions of related modules and are not interrupted by main functions of related modules.

The EthTSyn related modules are the EthIf and StbM modules.

**Caution**

Consider main function expectations when using preemptive tasks.

5 API Description

For an interface overview please see Figure 2-2.

5.1 Type Definitions

The types defined by the EthTSyn are described in this chapter.

Type Name	C-Type	Description	Value Range
EthTSyn_TransmissionModeType	uint8	Transmission mode	ETHTSYN_TX_ON, ETHTSYN_TX_OFF
EthTSyn_SyncStateType	enum	Synchronization state	ETHTSYN_SYNC ETHTSYN_UNSYNC ETHTSYN_UNCERTAIN ETHTSYN_NEVERSYNC

Table 5-1 Type definitions

5.2 Services provided by EthTSyn

5.2.1 EthTSyn_GetVersionInfo

Prototype	
void EthTSyn_GetVersionInfo (ETHTSYN_P2VAR(Std_VersionInfoType) VersionInfoPtr)	
Parameter	
VersionInfoPtr [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return Code	
void	none
Functional Description	
Returns the version information. EthTSyn_GetVersionInfo() returns version information, vendor ID and AUTOSAR module ID of the component.	
Particularities and Limitations	
-	
Call Context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-2 EthTSyn_GetVersionInfo

5.2.2 EthTSyn_Init

Prototype	
void EthTSyn_Init (EthTSyn_ConfigType* CfgPtr)	
Parameter	
CfgPtr [in]	Pointer to configuration
Return Code	
void	none
Functional Description	
Initialization of the EthTSyn module.	
Particularities and Limitations	
The function EthTSyn_InitMemory() must be called first	
Call Context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-3 EthTSyn_Init

5.2.3 EthTSyn_InitMemory

Prototype	
void EthTSyn_InitMemory (void)	
Parameter	
void	none
Return Code	
void	none
Functional Description	
Memory initialization of the EthTSyn module. Service to initialize module global variables at power up. This function initializes the variables in *_INIT_* sections. Used in case they are not initialized by the startup code.	
Particularities and Limitations	
-	
Call Context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-4 EthTSyn_InitMemory

5.2.4 EthTSyn_MainFunction

Prototype	
void EthTSyn_MainFunction (void)	
Parameter	
void	none
Return Code	
void	none
Functional Description	
Handles cyclic tasks of the EthTSyn module.	
Particularities and Limitations	
-	
Call Context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-5 EthTSyn_MainFunction

5.2.5 EthTSyn_SwtMgmt_MainFunction

Prototype	
void EthTSyn_SwtMgmt_MainFunction (void)	
Parameter	
void	none
Return Code	
void	none
Functional Description	
Handles the switch management Rx/Tx management objects used by the EthTSyn and processes received messages.	
Particularities and Limitations	
-	
Call Context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-6 EthTSyn_SwtMgmt_MainFunction

5.2.6 EthTSyn_SetTransmissionMode

Prototype	
void EthTSyn_SetTransmissionMode (uint8 CtrlIdx, EthTSyn_TransmissionModeType Mode)	
Parameter	
CtrlIdx [in]	EthIf controller index
Mode [in]	Mode that shall be applied: ETHTSYN_TX_OFF - turn off EthTSyn Tx capability ETHTSYN_TX_ON - turn on EthTSyn Tx capability
Return Code	
void	none
Functional Description	
Set the EthTSyn transmission mode. This API is used to turn on and off the TX capabilities of the EthTSyn.	
Particularities and Limitations	
-	
Call Context	
> TASK ISR2 > This function is Synchronous > This function is Non-Reentrant	

Table 5-7 EthTSyn_SetTransmissionMode

5.3 Services used by EthTSyn

In the following table services provided by other components, which are used by the EthTSyn are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
Crc	Crc_CalculateCRC8H2F()
Det	Det_ReportError()
Det	Det_ReportRuntimeError()
EthIf	EthIf_EnableEgressTimestamp()
EthIf	EthIf_GetCurrentTime()
EthIf	EthIf_GetEgressTimestamp()
EthIf	EthIf_GetIngressTimestamp()
EthIf	EthIf_GetRxMgmtObject()
EthIf	EthIf_GetTxMgmtObject()
EthIf	EthIf_ProvideTxBuffer()

Component	API
EthIf	EthIf_SetSwitchMgmtInfo()
EthIf	EthIf_SwitchEnableTimeStamping()
EthIf	EthIf_SwitchSetCorrectionTime()
EthIf	EthIf_Transmit()
IpBase	IpBase_GetUint16()
IpBase	IpBase_GetUint24()
IpBase	IpBase_GetUint32()
IpBase	IpBase_GetUint48()
IpBase	IpBase_GetUint64()
IpBase	IpBase_GetUint8()
IpBase	IpBase_PutUint16()
IpBase	IpBase_PutUint24()
IpBase	IpBase_PutUint32()
IpBase	IpBase_PutUint48()
IpBase	IpBase_PutUint64()
IpBase	IpBase_PutUint8()
StbM	StbM_BusGetCurrentTime()
StbM	StbM_BusSetGlobalTime()
StbM	StbM_EthSetMasterTimingData()
StbM	StbM_EthSetPdelayInitiatorData()
StbM	StbM_EthSetPdelayResponderData()
StbM	StbM_EthSetSlaveTimingData()
StbM	StbM_GetCurrentTime()
StbM	StbM_GetCurrentVirtualLocalTime()
StbM	StbM_GetOffset()
StbM	StbM_GetTimeBaseStatus()
StbM	StbM_GetTimeBaseUpdateCounter()
VStdLib	VStdLib_MemClr()
VStdLib	VStdLib_MemCpy()

Table 5-8 Services used by the EthTSyn

5.4 Callback Functions

This chapter describes the callback functions that are implemented by the EthTSyn and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `EthTSyn_Cbk.h` by the EthTSyn.

5.4.1 EthTSyn_RxIndication

Prototype	
<pre>void EthTSyn_RxIndication (uint8 CtrlIdx, Eth_FrameType FrameType, boolean IsBroadcast, ETHTSYN_P2CONST(uint8) PhysAddrPtr, ETHTSYN_P2VAR(uint8) DataPtr, uint16 LenByte)</pre>	
Parameter	
CtrlIdx [in]	EthIf controller index
FrameType [in]	Frame type of received Ethernet frame
IsBroadcast [in]	Broadcast indication: FALSE - frame is not a broadcast frame TRUE - frame is a broadcast frame
PhysAddrPtr [in]	Pointer to the Physical source address (MAC address in network byte order) of received Ethernet frame
DataPtr [in]	Pointer to payload of the received Ethernet frame (i.e. Ethernet header is not provided)
LenByte [in]	Length of received data
Return Code	
void	none
Functional Description	
Processing of received EthTSyn frames. By this API service the EthTSyn gets an indication and the data of a received frame.	
Particularities and Limitations	
-	
Call Context	
> TASK ISR2 > This function is Synchronous > This function is Non-Reentrant	

Table 5-9 EthTSyn_RxIndication

5.4.2 EthTSyn_TxConfirmation

Prototype	
void EthTSyn_TxConfirmation (uint8 CtrlIdx, uint8 BufIdx)	
Parameter	
CtrlIdx [in]	EthIf controller index
BufIdx [in]	Index of the buffer resource
Return Code	
void	none
Functional Description	
Confirms the transmission of an Ethernet frame. This callback function is called by lower layer (EthIf) if a message has been transmitted by the hardware.	
Particularities and Limitations	
-	
Call Context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-10 EthTSyn_TxConfirmation

5.4.3 EthTSyn_TrcvLinkStateChg

Prototype	
<code>void EthTSyn_TrcvLinkStateChg (uint8 CtrlIdx, EthTrcv_LinkStateType TrcvLinkState)</code>	
Parameter	
CtrlIdx [in]	Index of the Ethernet controller
TrcvLinkState [in]	New link state of the transceiver ETHTRCV_LINK_STATE_DOWN - Link Down ETHTRCV_LINK_STATE_ACTIVE - Link Up
Return Code	
void	none
Functional Description	
Callback function that notifies a changed state of the transceiver link. Allows resetting state machine in case of unexpected Link loss to avoid inconsistent Sync and Follow_Up sequences	
Particularities and Limitations	
-	
Call Context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-11 EthTSyn_TrcvLinkStateChg

5.5 Configurable Interfaces

5.5.1 Notifications

At its configurable interfaces the EthTSyn defines notifications that can be mapped to callback functions provided by other modules. The mapping is not statically defined by the EthTSyn but can be performed at configuration time. The function prototypes that can be used for the configuration have to match the appropriate function prototype signatures, which are described in the following sub-chapters.

5.5.1.1 SyncSentCbK

Prototype	
<code>void EthTSyn_SyncSentCbK (EthTSyn_IntPortIdxType PortIdx, const uint8* TxBufPtr)</code>	
Parameter	
PortIdx[in]	Index of the EthTSyn port the sync message was transmitted on
TxBufPtr[in]	Pointer to the Ethernet transmission buffer of the transmitted sync message (Network-Byte-Order)
Return Code	
void	none

Functional Description
This callback function can be used for optional notification of a transmitted sync message.
Particularities and Limitations
-
Call Context
> Interrupt

Table 5-12 SyncSentCbK

5.5.1.2 FollowUpSentCbK

Prototype	
void EthTSyn_FollowUpSentCbK (EthTSyn_IntPortIdxType PortIdx, const uint8* TxBufPtr)	
Parameter	
PortIdx[in]	Index of the EthTSyn port the follow up message was transmitted on
TxBufPtr[in]	Pointer to the Ethernet transmission buffer of the transmitted follow up message (Network-Byte-Order)
Return Code	
void	none
Functional Description	
This callback function can be used for optional notification of a transmitted follow up message	
Particularities and Limitations	
-	
Call Context	
> Interrupt	

Table 5-13 FollowUpSentCbK

5.5.1.3 SwitchSyncStateChangeCbK

Prototype	
void EthTSyn_SwitchSyncStateChgCbK (uint8 EthTSynSwitchIdx, EthTSyn_SyncStateType SyncState)	
Parameter	
EthTSynSwitchIdx[in]	Index of the ethernet switch (cascade) in the context of the EthTSyn
SyncState[in]	The synchronization state of the passed switch (cascade)
Return Code	
void	none

Functional Description
This callback function can be used for optional notification about changes in the synchronization state of an ethernet switch (cascade).
Particularities and Limitations
-
Call Context
> Task

Table 5-14 SwitchSyncStateChangeCbK

6 Configuration

The EthTSyn attributes can be configured and generated with the tool DaVinci Configurator Pro.

6.1 Configuration Variants

The EthTSyn supports the configuration variants

> VARIANT-PRE-COMPILE

The configuration classes of the EthTSyn parameters depend on the supported configuration variants. For their definitions please see the EthTSyn_bswmd.arxml file.

6.2 Configuration of switch time synchronization

Figure 6-1 shows an overview of the switch time synchronization configuration for one switch (cascade). For each switch cascade such a configuration is required. A short explanation for the highlighted configuration options is provided in the following paragraphs:

- 1 Choose whether rate ratio correction should be applied to slave switches or not.
- 2 Reference to the (Master-)Switch which should be synchronized (in context of the EthIf).
- 3 Optional reference to the management port of the Master-Switch. This reference is only used when the Host CPU is acting as time master. With this reference it is possible to enable a separate switch sync frame which is used for the synchronization of the ethernet switch.
- 4 Automatically calculated index of the EthTSyn switch time sync cascade. This index is provided as symbolic name value and is e.g. used in the optional switch sync state change notification.
- 5 Configuration of the time synchronization parameters, e.g. the time domain used for synchronization and the optional switch sync state change notification callback.
- 6 Optional reference(s) to the Slave-Switch(s) of the cascade (in context of the EthIf)
- 7 The used synchronization mechanism:
 - 7.1 RateRegulator: Optionally offset is corrected by time leap before synchronization was established the first time. Afterwards only rate ratio is used to compensate frequency differences and to reduce the offset. For a detailed description of this mechanism and its configuration parameters see chapter 3.1.13.3. An overview of the rate regulator configuration is shown in Figure 6-2.
 - 7.1.1 Number of parallel rate measurements. This is also the duration of each rate measurement when this duration is measured in number of synchronization cycles.
 - 7.1.2 Duration over which the offset shall be reduced to zero by rate regulation. This duration is measured in number of synchronization cycles.
 - 7.1.3 If the current offset exceeds this optional threshold, it will be corrected by time leap instead of rate regulation.

- 7.1.4 Configuration of the maximum and minimum allowed rate ratio correction. These values are always relative to the last rate ratio.
- 7.1.5 Choose whether the initial offset correction by time leap should be used or not.



Note

For a more detailed explanation of the different configuration parameters please refer to their description provided in EthTSyn_bswmd.arxml.

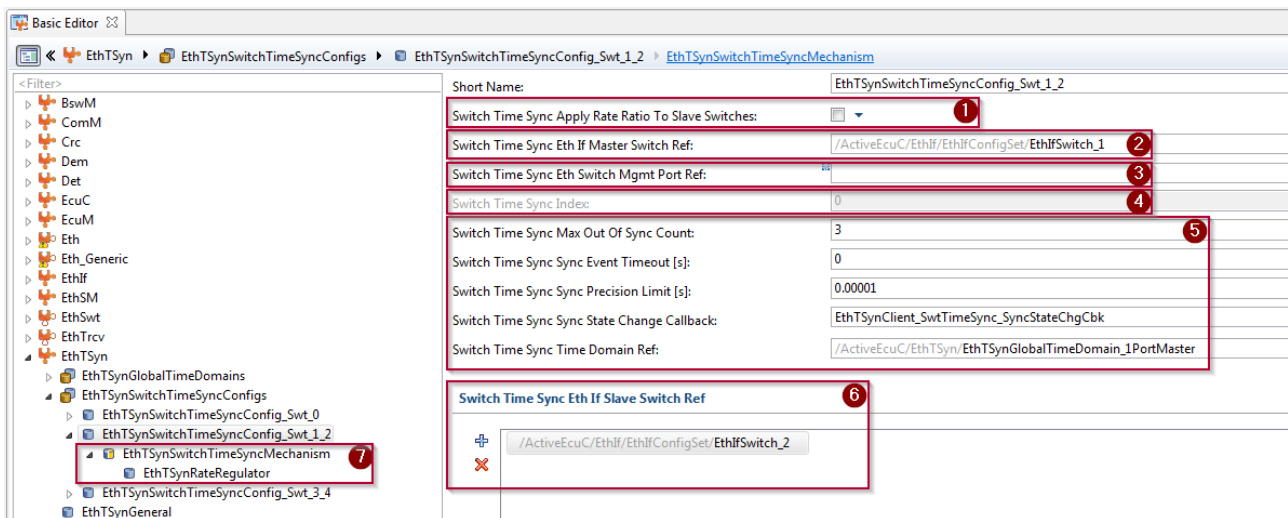


Figure 6-1 Configuration of switch time synchronization

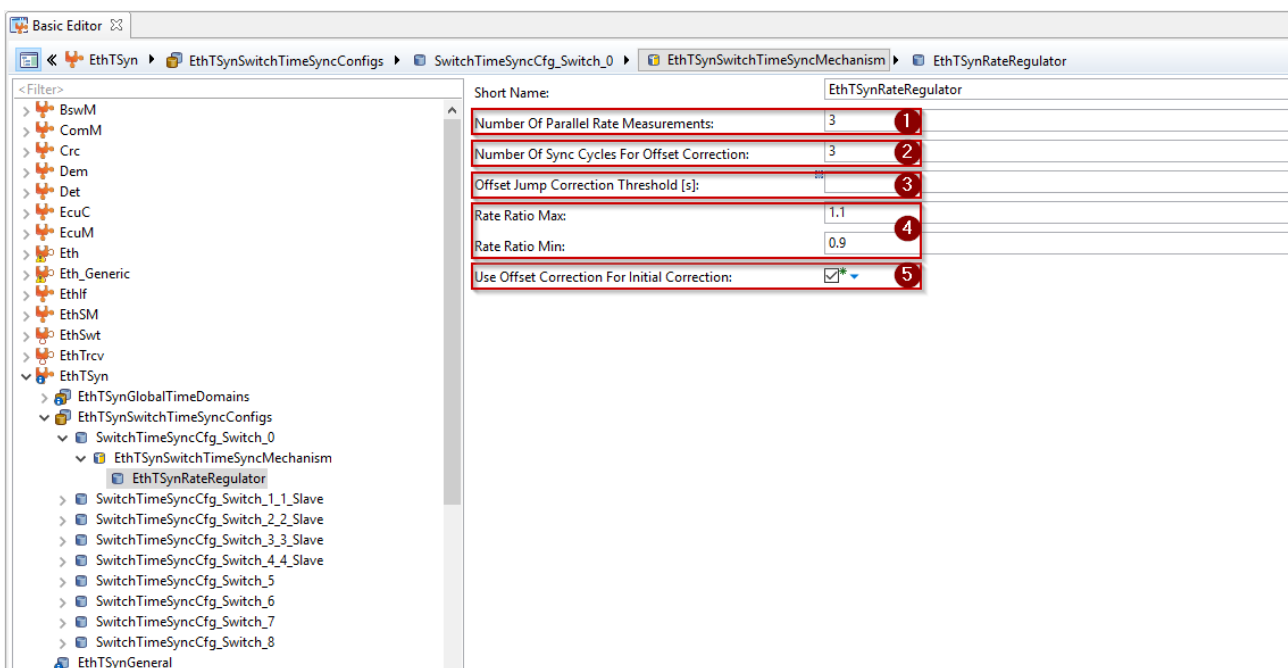


Figure 6-2 Configuration of rate regulator

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
Configurator Pro	DaVinci Configurator Pro 5 generation tool for MICROSAR components

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
Eth	Ethernet Driver
EthIf	Ethernet Interface
EthSwt	Ethernet Switch
EthTSyn	Time Synchronization over Ethernet
gPTP	Generalized Precision Time Protocol
HW	Hardware
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
MII	Media Independent Interface
Pdelay	Path Delay
PdelayReq	Pdelay request
PdelayResp	Pdelay response
PdelayRespFollowUp	Pdelay response follow up
StbM	Synchronized Time-Base Manager
SW	Software
SWS	Software Specification
TSN	Time-Sensitive Networking

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com